JavaScript 1.9

Event Loop & Scheduling

Print-first interview refresher | execution → queues → rendering opportunities

Ask two questions:
what is ready to run?
what must finish first?

1 | Readiness does not interrupt a running call

Call stack
ordinary calls run to completion

→

Host environment
timers / network / input

→

Ready callback
wait for its scheduling boundary

A zero-delay timer does not run inline. A synchronous callback is not a new task.

Browsers have multiple task sources; no universal FIFO order across input, timers and network.

2 | Microtask checkpoint: drain, including newly queued work

Ready Promise reactions and queueMicrotask follow enqueue order; no fixed Promise-first rule.

Promise executor: synchronous. A reaction on a pending promise is not ready yet.

sync → end → promise → micro → nested → timer

If promise adds nested while micro already waits, nested joins the tail.

A timer callback can queue a microtask that runs before the next timer callback.

3 | An empty stack is not enough if the checkpoint never ends

Recursive microtasks
No growing stack, but tasks can starve.
Awaiting ready promises is not a UI yield.

Bounded chunks in ordinary tasks
Return control between short batches.
One costly item can still block the thread.

Create a scheduling opportunity, not a promise that every intermediate state will be painted.

4 | Timers: eligibility threshold, not an appointment

Busy JavaScript, other tasks and background policy can delay delivery.

Nested timer level > 5 and delay < 4 ms: raise the delay to 4 ms; not an exact 4 ms period.

Count time from a deadline, not from the number of interval callbacks.

`remaining seconds = max(0, ceil((deadline - now) / 1000))` | use compatible clocks

5 | requestAnimationFrame: one visual callback, then request again

Use its timestamp: position = speed × elapsed time. Do not add fixed pixels per frame.

Coalesce updates: latest value + at most one pending frame; clear the ID before applying.

Cancel the pending frame on disposal. Hidden pages may pause animation callbacks.

A long callback delays rendering. rAF is not an after-paint notification.

6 | Predict causes, verify the contract

Name the runtime → execute sync code → enqueue only ready work → drain microtasks.

Continue with permitted tasks and frame work; keep unspecified cross-source order partial.

Test cancellation, empty input, reentrant updates and independent owners.

Use controlled timestamps for math and real browser APIs for lifecycle; avoid exact milliseconds.

Rendering is not guaranteed after every task

DOM state, layout and displayed pixels are different. Node.js has its own host scheduling model.